Akshay Parte

+91-7021289701 | akshayparte580@gmail.com | linkedin.com/in/akshayparte/

SKILLS

Languages: JavaScript(ES6), TypeScript, Python
Frontend: React, HTML, CSS, Material UI
Backend: Node.js (Express, NestJS), REST APIs, GraphQL, Server-Sent Events (SSE)
Databases: MySQL, Redis
Messaging & Streaming: Kafka, RabbitMQ
DevOps & Tools: Docker, Git, GitHub, Bitbucket, Linux, Apache (Reverse Proxy), CI/CD Basics
Workflow & Practices: Jira, Agile/Scrum

EXPERIENCE

Clarapth *Mumbai, India*
Full Stack Engineer, October 2023 – December 2025

- Developed and maintained the **Console application** using React, TypeScript, and Material UI for controlling medical testing machines and displaying session and diagnostic data.
- Developed **REST and GraphQL APIs** using Node.js, NestJS, and Express.js to support communication between frontend applications, backend services, and machine-related systems.
- **Replaced continuous frontend API polling with Server-Sent Events (SSE)** for real-time machine statistics, enabling the backend to push updates when new data was available and reducing unnecessary API requests.
- **Implemented event-driven data synchronisation using MySQL event listeners and Kafka** to synchronise machine session data between the Console and Session applications. Kafka provided message persistence, allowing consumers to resume processing after temporary downtime.
- **Improved database consistency and controlled database access** by restructuring database operations, adding appropriate indexes, and introducing a dedicated Database Access Layer (DAL) service. Centralised queries and transactions through controlled APIs to ensure database changes were reviewed and managed consistently across teams.
- **Automated machine deployment and validation processes** by creating scripts for environment initialisation, database setup, machine-specific configuration, service startup, and health checks. Added producer-consumer validation to verify Kafka communication during deployment.
- **Migrated application deployments from PM2 to Docker**, packaging application dependencies and configuration into reusable container images to provide consistent deployments across different machines and reduce manual setup.
- **Refactored complex React components into reusable components and improved state management and loading states**, resolving UI rendering and flickering issues while providing better feedback during asynchronous API operations.

Tantrasoft Solutions, *Mumbai, India*

Full Stack Developer, November 2021 – April 2023

- Developed and maintained backend services using **Node.js and Express.js** for a pharma-tech reporting application, supporting data processing, persistence, and report generation.
- **Implemented a UDP-based data ingestion service** to receive real-time data from external sources and persist it into MySQL, enabling the application to process incoming data without relying on manual data entry.
- **Built REST APIs to process and expose stored data** to frontend applications, enabling generation of medicine-related reports from centralised backend data.
- **Improved data reliability in the reporting workflow** by handling incoming data through the backend before exposing processed information to the frontend, reducing inconsistencies between raw input and generated reports.
- **Resolved frontend-backend integration issues** by working with React and Angular developers on API contracts, data handling, UI integration, and application bugs.
- **Participated in end-to-end feature development**, from requirement analysis and backend implementation through debugging, QA validation, and production delivery.

PROJECTS

Reverse Proxy Server

Tech Stack: Apache, Node.js, NestJS, Express.js, React.js, TypeScript, JavaScript (ES6), MySQL, Docker, Redis

- **Solved the need to expose multiple internal applications through separate public ports** by implementing an **Apache reverse proxy** with unified URL-based routes such as `/console` and `/histostation`.
- **Removed direct exposure of application ports** by keeping services running on local ports and routing external traffic through controlled Apache proxy routes, improving access control and reducing direct access to internal services.
- **Implemented deployment maintenance mode** using Apache configuration flags to redirect users to a static downtime page while application updates were being deployed.
- **Improved deployment control by managing application traffic at the proxy layer**, allowing services to be taken offline during deployments without requiring changes to the individual applications.

DAL (Database Access Layer) Server

Tech Stack: Node.js, NestJS, Prisma, MySQL, Kafka, Docker

- **Solved inconsistent and uncontrolled database changes across multiple internal teams** by building a centralised **Database Access Layer (DAL)** service for applications interacting with shared databases.
- **Centralised database operations through controlled APIs** for create, update, delete, and fetch operations, ensuring database queries and transactions were implemented consistently instead of being directly executed by individual applications or teams.
- **Introduced a review-based database change workflow**, where database queries and schema-related changes were reviewed and approved before being deployed through the DAL, reducing the risk of incorrect data modifications during machine testing.
- Implemented **input validation and transaction handling** across DAL APIs to prevent invalid data operations and maintain consistency during database updates.
- Used **Prisma with MySQL** to manage database operations and **Kafka** for asynchronous inter-service communication and background task processing.
- Added **database monitoring and background task handling** to track database activity and execute operations based on application requirements.

SSE Integration for Real-Time Console Updates

Tech Stack: Node.js, NestJS, React.js, Redis, MySQL, ROS

- **Solved excessive frontend polling** by replacing APIs that were called every 2 seconds with **Server-Sent Events (SSE)** for real-time machine statistics and analytics updates.
- **Integrated SSE with the ROS client** so that when new machine data was received, the backend could immediately push updated statistics to the React console instead of waiting for the next polling cycle.
- **Synchronized Redis cache updates with SSE event publishing** to keep frequently changing machine and analytics data consistent across services.
- Used **Redis for fast access to frequently updated machine status and analytics data**, reducing the need for repeated database reads.
- **Improved real-time dashboard responsiveness and reduced unnecessary API requests** by moving from periodic polling to event-driven updates.

Data Processing & Automation Service

Tech Stack: Python, MySQL, Docker, Linux, Cron Jobs

- **Automated repetitive data processing and reporting tasks** using Python scripts to transform application data and store structured results in MySQL, reducing manual operational effort.
- **Built scheduled background jobs** using Cron for report generation, data validation, and file-processing workflows, allowing recurring tasks to run automatically without manual intervention.
- **Implemented CSV and JSON data ingestion pipelines** to process data received from external systems, validate and transform it into structured formats for internal applications.
- **Added data validation and transformation steps before database persistence** to improve consistency and reduce invalid or incorrectly formatted data entering MySQL.
- **Containerised the automation services using Docker** to provide a consistent execution environment across development and production machines.

Education

Bachelor of Commerce (B.Com) – Graduation
P. D. Karkhanis College of Arts & Commerce
University of Mumbai | 2020